

TIPO A

Calificación	
--------------	--

DEPARTAMENTO DE INFORMÁTICA

Curso: 2º Desarrollo de Aplicaciones Web

Módulo: Desarrollo Web Entorno Cliente

Examen: Examen 2ª Evaluación – JavaScript

Fecha: 3/02/2026

Nombre:

Desarrolla el contenido. (**Total 10 puntos**). Tiempo estimado: **120 minutos**.

Unidades 4, 5 y 6: DOM, BOM, Eventos y Asincronía

Nombre del Proyecto: Tech-Store Dinámica

Tiempo Máximo: 120 minutos

Puntuación: Bloque Base (Máx. 8 puntos) + Bloque Excelencia (Máx. 2 puntos)

Entrega: Comprime en un .zip los archivos index.html, style.css, productos.json y app.js. Asegúrate de que no haya rutas absolutas en el fetch.

1. CONTEXTO

Debes desarrollar la lógica de una tienda tecnológica. La empresa te facilita la estructura HTML y CSS, pero tú debes programar el comportamiento dinámico. Los datos de los productos deben obtenerse de un archivo externo productos.json.

2. REQUISITOS TÉCNICOS

Para asegurar la calidad del código, tu archivo app.js **debe** seguir esta estructura de nombres:

- 1) **Variables Globales:** articulosCarrito (array), productosDB (array), listaProductos (contenedor).
- 2) **Funciones Obligatorias:**
 - a) obtenerProductos(): Carga asíncrona mediante Fetch.
 - b) pintarCards(datos): Renderizado de productos en el DOM.
 - c) limpiarHTML(nodo): Borrado eficiente de nodos (uso de while).
 - d) gestionarCarrito(e): Lógica de añadir/restar usando delegación de eventos.
 - e) sincronizarStorage(): Persistencia de datos.

3. BLOQUE BASE (Máximo 8 puntos)

- **Tarea 1: Asincronía y Carga (2 Ptos)** Carga los datos de productos.json usando async/await. Mientras se cargan, debe ser visible el elemento #loading. Implementa gestión de errores con try/catch.
- **Tarea 2: Renderizado y Filtrado (2 Ptos)** Muestra los productos dinámicamente. Implementa un filtro por categoría que actualice la vista sin recargar la página. Usa la función de limpieza eficiente vista en la Actividad 5.4.

- **Tarea 3: El Carrito y Delegación (2 Ptos)** Permite añadir productos (sumando cantidad si ya existen) y restar unidades. Es obligatorio el uso de **Delegación de Eventos** en los contenedores padres.
- **Tarea 4: Persistencia y Buenas Prácticas (2 Ptos)** El carrito debe recuperarse al recargar (localStorage). El código debe estar correctamente indentado, usar nombres semánticos y contener comentarios explicativos (10% de la nota).

4. BLOQUE DE EXCELENCIA (Hasta +2 puntos)

- **Reto A: Gestión de Stock Dinámico (+1 Pto)** Cada vez que se añada un producto, resta una unidad de su stock en el array `productosDB`. Si llega a 0, deshabilita el botón, cambia su texto a "AGOTADO" y aplica la clase CSS `.agotado` a la card.
- **Reto B: Oferta Flash (BOM) (+1 Pto)** Implementa un temporizador de 60 segundos al cargar la página. Si el usuario interactúa con el carrito antes de que expire, aplica un 10% de descuento al total. Muestra un mensaje dinámico indicando el estado de la oferta.

GUÍA DE IMPLEMENTACIÓN DE LA PRÁCTICA: TECHSTORE

1. VARIABLES GLOBALES Y SELECTORES

Antes de crear funciones, necesitamos definir la "mesa de trabajo".

```
// Para los datos (Estado)
let productosDB = []; // Base de datos propia local con los datos del JSON
let articulosCarrito = []; // La lista de la compra del usuario
// let articulosCarrito = JSON.parse(localStorage.getItem('carrito')) || [];

// B. Selectores. Aquí obtenemos las referencias al HTML
const listaProductos = document.querySelector('#lista-productos');
const carritoContenedor = document.querySelector('#carrito-items');
const precioTotalHTML = document.querySelector('#precio-total');
const filtroCategoria = document.querySelector('#filtro-categoria');
const contadorHeader = document.querySelector('#contador-productos')
```

2. MAPA DE FUNCIONES FLUJO DE TRABAJO

- **obtenerProductos():** Se encarga de la comunicación con el exterior (Fetch).
- **pintarCards():** Se encarga de transformar los datos en HTML visual.
- **limpiarHTML():** Su única misión es borrar el rastro anterior para evitar duplicados.
- **gestionarCarrito():** Escucha los clics y decide si debe añadir o quitar algo del array del carrito.
- **actualizarCarritoHTML():** Sincroniza lo que hay en el array con lo que se ve en la pantalla.
- **sincronizarStorage():** Guarda la "foto" actual del carrito en el navegador para que no se destruya el carrito y las cantidades de productos al hacer F5.

3. FASES DEL DESARROLLO

Fase 1: Carga de Datos asíncrona (Fetch)

En primer lugar mostramos el elemento: #loading al inicio (no mostrado en el código). Luego traemos la información del archivo JSON (sería igual desde una DB externa). Usamos `await fetch('productos.json')` y `await respuesta.json()`. Ocultamos el #loading (con `display: none`) después de recibir los datos. Guardamos el resultado en el array `productosDB`. Y, finalmente, llamamos a la función para pintar las Cards: `pintarCards(productosDB)`

```
async function obtenerProductos() {
  try { // TODO...
    const respuesta = await fetch('productos.json');
    productosDB = await respuesta.json();
    pintarCards(productosDB); // Una vez cargados, los mostramos
  } catch (error) {
    console.error("Error al cargar los datos:", error);
  }
}
```

Fase 2: Renderizado (DOM)

Transformamos los objetos del array en tarjetas visuales (Cards de productos: clase `.producto-card` del CSS) que mostramos al usuario. Para ello usaremos un bucle `forEach` y `createElement`. Es importante que cada botón “Añadir” lleve asociado un atributo `data-id` que es el ID único del producto para saber qué producto se ha pulsado.

```
function pintarCards(lista) {
  limpiarHTML(listaProductos); // 1. Borramos lo anterior

  lista.forEach( producto => {
    const card = document.createElement('div');
    card.classList.add('producto-card');
    card.innerHTML = `
      <h3>${producto.nombre}</h3>
      <p>${producto.precio}€</p>
      <button class="btn-add" data-id="${producto.id}">Añadir</button>
    `;
    listaProductos.appendChild(card);
  });
}
```

Para el reto de excelencia: Si `producto.stock === 0`, añade la clase `.agotado` a la card, cambia el texto del botón a "AGOTADO" y ponle el atributo `disabled`. Así evitaras compras sin stock.

Fase 3: Delegación de Eventos

Capturamos el clic de forma eficiente: Se trata de no poner un evento a cada botón (imagina que tuviéramos 1000 productos); sino que se lo ponemos al contenedor padre (`#listaProductos`), o sea, escucharemos los clics en el contenedor padre.

Luego, recuperamos el ID del producto con el método: `getAttribute('data-id')`. Este devuelve un string que convertimos en número con `parseInt()`.

```
// Listener principal
listaProductos.addEventListener('click', gestionarClickProductos);

function gestionarClickProductos(e) {
  // Comprobamos si el elemento clicado tiene la clase del botón de añadir
  if (e.target.classList.contains('btn-add')) {

    // Leemos el valor del atributo data-id
    const idString = e.target.getAttribute('data-id');

    // Convertimos a número para poder comparar
    const idNumb = parseInt(idString);

    // Pasamos el ID a la lógica de añadir
    agregarAlCarrito(idNumb);
  }
}
```

Fase 4: Lógica del Carrito (Procesamiento)

Gestionamos la lista de compra sin duplicar elementos. Si el producto existe, podemos usar la función `.map()` para localizarlo por su ID y sumar 1 a su cantidad.

Si el producto no está en la lista del carrito, usamos la función `.find()` para localizar sus datos en la DB (`productosDB`) y añadirlo al carrito con `cantidad=1`.

```
function agregarAlCarrito(id) {  
  // 1. ¿Ya existe en el carrito?  
  const existe = articulosCarrito.some( prod => prod.id === id );  
  
  if (existe) {  
    // TODO. Actualizamos cantidad (usa .map)  
  } else {  
    // TODO. Buscamos el producto en productosDB y lo añadimos con cantidad: 1  
  }  
  actualizarCarritoHTML(); // Refrescamos la vista del carrito  
}
```

Fase 5: Totales y Persistencia

Cálculos finales y `LocalStorage`.

```
function actualizarCarritoHTML() {  
  limpiarHTML(carritoContenedor);  
  
  // Calculamos el total usando reduce  
  const total = articulosCarrito.reduce((acc, item) => acc + (item.precio *  
item.cantidad), 0);  
  precioTotalHTML.textContent = `${total.toFixed(2)}€`;  
  
  sincronizarStorage();  
}  
  
function sincronizarStorage() {  
  localStorage.setItem('carrito', JSON.stringify(articulosCarrito));  
}  
  
// Función para vaciar los contenedores antes de volver a pintar: sustituye a innerHTML=""  
function limpiarHTML(contenedor) {  
  while (contenedor.firstChild) {contenedor.removeChild(contenedor.firstChild);}  
}
```

3. DIAGRAMA DE FLUJO LÓGICO

¿Qué pasa cuando haces clic en "Añadir"?

1. **Evento:** Se detecta el clic en el botón.
2. **Identificación:** Obtenemos el data-id (ej: "10").
3. **Lógica:** Se busca el producto y se actualiza el array `articulosCarrito`.
4. **Sincronización:** Se llama a `actualizarCarritoHTML()` y `sincronizarStorage()`.
5. **Vista:** El usuario ve el nuevo producto y el nuevo precio total al instante.

4. CONSEJOS

- Siempre llamamos a limpiarHTML() antes de pintar las Cards para que los productos no se acumulen infinitamente. Esta **función limpiarHTML**: Debe contener un bucle while(contenedor.firstChild) para ser eficiente.
- **Conversión de tipos**: Lo que viene del HTML es un String. Si comparas "1" === 1 dará falso. Para evitarlo usa parseInt().
- **Depuración**: Usa console.log(articulosCarrito) después de cada clic para verificar que los datos cambian correctamente antes de intentar pintarlos.
- Cada vez que cambies el array articulosCarrito (añadir, quitar o vaciar), debes llamar a la función que **pinta el carrito** y a la que **guarda en el Storage**.
- **Inicialización**: Al cargar el documento (DOMContentLoaded), no olvides leer el localStorage para que el carrito no empiece de cero si el usuario refresca la página.

5. RÚBRICA DE EVALUACIÓN

Criterio	Logrado (100%)	En Proceso (50%)	No Logrado (0%)
Asincronía (Fetch)	Usa async/await y try/catch correctamente. Gestiona el estado de carga.	Realiza el fetch pero falla la gestión de errores o el indicador de carga.	No utiliza fetch o el código genera errores críticos.
DOM y Filtrado	Renderizado dinámico fluido y filtro funcional. Limpieza eficiente con while.	Renderiza pero el filtro falla o usa métodos ineficientes (innerHTML = ").	No renderiza dinámicamente los productos.
Carrito y Eventos	Lógica de añadir/restar perfecta. Usa delegación de eventos correctamente.	El carrito funciona pero no usa delegación o falla al actualizar cantidades.	No hay lógica de carrito funcional.
Persistencia y Calidad	localStorage funcional. Código limpio, comentado e indentado.	Fallos leves en persistencia o código difícil de leer/sin comentarios.	No usa LocalStorage y el código es desordenado.
Excelencia (Extra)	Implementa stock dinámico y/o temporizador BOM sin errores de lógica.	Intenta los retos pero presentan errores de cálculo o de sincronización visual.	No intenta los retos o el código añadido rompe la base.

GUÍA DE IMPLEMENTACIÓN: TECHSTORE

Para este examen, debéis estructurar vuestro código en el archivo app.js siguiendo este esquema. Se evaluará el uso de los nombres de funciones y variables propuestos.

1. Variables Globales y Selectores

Declarad al inicio los selectores necesarios para interactuar con el HTML:

- `const listaProductos`: Contenedor donde se pintarán las cards.
- `const carritoContenedor`: El `<ul>` o contenedor del carrito lateral.
- `let articulosCarrito = []`: Array para los productos elegidos (iniciadlo con lo que haya en `localStorage`).
- `let productosDB = []`: Array para guardar los datos que vienen del archivo JSON.

2. Estructura de Funciones

A. Carga de Datos

Función: `async function obtenerProductos()`

- Debe realizar la petición `fetch` al archivo `productos.json`.
- Usad `await` para esperar la respuesta y el `.json()`.
- **Obligatorio:** Envolverlo en un bloque `try { ... } catch (error) { ... }`.

B. Renderizado (Unidad 4)

Función: function pintarCards(datos)

- Recibe un array y utiliza un bucle (forEach) para crear el HTML de cada producto.
- **Importante:** Antes de pintar, llama a limpiarHTML(listaProductos).

Función: function limpiarHTML(contenedor)

- Tal como hiciste en la **Actividad 5.4**, usa un bucle while(contenedor.firstChild) para borrar los hijos de forma eficiente.

C. La Delegación de Eventos

Para gestionar los clics en botones que se crean dinámicamente, usad un solo listener en el padre.

Función: function gestionarClickProductos(e)

- Asociadla al evento click de listaProductos.
- Usad e.target.classList.contains('btn-add') para saber si se pulsó el botón.
- Leed el ID del producto con e.target.getAttribute('data-id').

D. Lógica del Carrito

Función: function actualizarCarrito()

- Recorre articulosCarrito y pinta los elementos en el lateral.
- Calcula el precio total y actualiza el contador del header.
- Llama a sincronizarStorage().

Función: function sincronizarStorage()

- Usa localStorage.setItem para que los datos persistan (visto en **Actividad 5.9**).

3. Hoja de guía

Si quieres...	Usa...	Recuerda que...
Convertir ID	parseInt()	El ID del atributo data-id es un String, conviértelo a número para comparar.
Buscar en Array	.find() o .some()	Úsalos para saber si un producto ya está en el carrito.
Eliminar/Filtrar	.filter()	Crea un nuevo array excluyendo el ID que quieres borrar.

Si quieres...	Usa...	Recuerda que...
Transformar	.map()	Ideal para actualizar la cantidad de un producto específico sin perder el resto.